Kuku AI Generalist Assignment: Find the Promos

Name: Saurav Raj | **Roll No:** 230941 | Department : Statistics and Data Science
IIT Kanpur

Repo:
https://github.com/sauravraj2003/kuku-promo-discovery-230941-Saurav-Raj-IITK
Deployed link: https://kuku-promo-discovery-230941-saurav-raj.onrender.com

1 Problem Choice & Core Assumptions

I chose Problem 1 to solve a critical operational bottleneck in the microdrama ecosystem. Manual promo discovery takes editors 4 to 6 hours per 50-episode show. My goal was to replace this with a sub-5-second automated engine prioritizing algorithmic rigor over subjective LLM hallucinations.

Core Assumptions:

1. **Duration:** Promos must be strictly bounded between 20.00 and 60.00 seconds.
2. **3-Second Hook:** To halt scrolling, a dramatic action or line must occur within the first 3 seconds.
3. **Dialogue Integrity:** A video cut must never fall mid-sentence. Audio flow is paramount.
4. **Evaluation:** Virality relies heavily on visual execution. I optimized for source grounding, narrative tension, and mechanical cut safety rather than uncalibrated AI "virality" scores.

2 Solution Architecture

Instead of asking an LLM to guess timestamps, I built a deterministic decision-support engine:

Screenplays → Exact Centisecond Parser → Dialogue Safety Index
↓
React Review Workbench ← SQLite Candidate Store ← Combinatorial Generator

1. **Centisecond Indexing (`parser.py`):** Floating-point timecode drifts are eliminated by converting all timestamps to integer centiseconds (e.g., `01:19.00` = 7900cs).
2. **Dialogue Safety (`timeline.py`):** Snaps cut points to verified screenplay pauses. Any cut occurring during active spoken dialogue is mathematically disqualified.
3. **Combinatorial Generator (`generator.py`):** Finds all legal 20 to 60s windows containing an opening hook and an unresolved cliffhanger. Discovered 23,602 valid candidates.
4. **Deduplication (`store.py`):** Clusters similar overlapping clips around anchor narrative beats to prevent UI clutter.
5. **Interactive Workbench (`frontend/`):** A FastAPI/React interface where editors instantly audition, filter, and validate top candidates.

3 Top Promo Recommendations

Filtering for verified hooks ($\leq$ 3s) and unresolved endings yielded these top distinct moments:

#	Ep	Timecode	Dur	Hook Quote (≤ 3s)	Cliffhanger Ending	Rationale
1	**2**	`00:00.00→00:51.40`	51.4s	*"I cannot marry you..."*	*"Why are you surprised...?"*	Immediate altar rejection and family betrayal.
2	**23**	`01:19.00→02:11.50`	52.5s	*"Someone is poisoning her."*	Door latch turns open	Race against time interrupted by approaching threat.
3	**10**	`00:09.10→00:54.00`	44.9s	*"Blood running down..."*	*"Does it hurt?"*	Visceral physical injury hook ending on emotional vulnerability.
4	**1**	`01:35.80→01:59.80`	24.0s	Close-up on Juliana rising	*"So that is what I was worth?"*	Fast melodrama with an explosive, unanswerable accusation.
5	**4**	`00:34.00→01:25.80`	51.8s	*"A woman betrayed."*	*"Are you certain?"*	High intrigue introducing an assassination plot observer.

4 System Performance & Limitations

How I Know It Works:

- **100% Dialogue Safety:** Zero candidates cut inside spoken words.
- **Deep Interior Discovery:** 98.7% of the 23,602 generated cliffhangers end mid-episode, proving the tool finds internal dramatic pivots, not just file endings.
- **Reliability:** 93 automated pytest unit/integration tests pass. Deterministic execution produces identical SHA-256 hashes every run.

Known Failure Modes:

- **False Positive Cliffhangers:** Mundane clarifying questions (e.g., *"What is for dinner?"*) are occasionally flagged as cliffhangers due to lexical punctuation rules (`?`).
- **False Positive Hooks:** Casual backstory exposition mentioning keywords (like *"secret"*) gets flagged even if the delivery lacks dramatic urgency.
- **Missed Silent Actions:** Tense physical moments without explicit text cues (dropping an object, a betraying glance) bypass text-based heuristics.

5 Production Economics

- **Speed:** A full 50-episode ingestion and discovery run takes just **4.1 seconds** on a standard CPU.
- **Cost:** $0 API cost for the deterministic pass. Optional LLM semantic scoring (Groq Llama 3) costs less than $0.02 per show.
- **Workflow Impact:** Compresses 4 to 6 hours of manual scrubbing into a 10-minute shortlist review.

6 AI Tools Used & Their Mistakes

- **Tools Used:** Gemini for React component authoring and test fixture scaffolding.
- **The Mistake:** Initially, I asked AI models to extract timestamps directly. They hallucinated non-existent fractional seconds (e.g., `00:41.33`) and repeatedly placed cuts in the middle of spoken sentences. This failure proved that timestamp boundary generation must be handled by strict Python deterministic rules, not LLMs.